

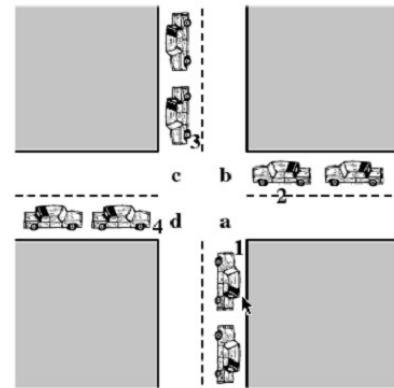
Deadlock

李旭东

leexudong@nankai.edu.cn

Nankai University

Case: Traffic

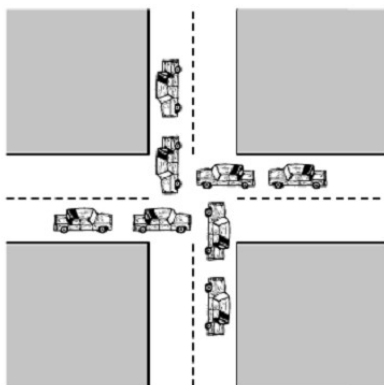


(a) Deadlock possible

leexudong@nankai.edu.cn

2

Case: Traffic



(b) Deadlock

leexudong@nankai.edu.cn

3

Objectives

- Deadlock Definition
- Deadlock Conditions
- Deadlock Modeling
- Deadlock Detection
- Deadlock Recovery
- Deadlock Avoidance
- Deadlock Prevention

leexudong@nankai.edu.cn

4

Deadlock

- Deadlock
 - A situation in which two or more competing actions are each waiting for the other to finish, and thus neither ever does
 - A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.
 - Deadly-Embrace 死锁

leexudong@nankai.edu.cn

5

The Reason of Deadlock

- Deadlock
 - Competing Resource
 - Running Order of a Set of Processes

leexudong@nankai.edu.cn

6

Resource

- Resource which causes deadlock
- hardware, software, information
- Preemptable, Nonpreemptable
 - 抢占式、非抢占式

leexudong@nankai.edu.cn

7

Nonpreemptable Resources

- Sequence of events required to use a resource:
 - Request the resource
 - Use the resource
 - Release the resource

leexudong@nankai.edu.cn

8

Resource Acquisition

```
typedef int semaphore;
semaphore resource_1;

void process_A(void) {
    down(&resource_1);
    use_resource_1();
    up(&resource_1);
}
```

(a)

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}
```

(b)

leexudong@nankai.edu.cn

9

Which situation will be a deadlock?

```
semaphore resource_1;
semaphore resource_2;
void process_A(void){
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}
void process_B(void){
    down(&resource_2);
    down(&resource_1);
    use_both_resources();
    up(&resource_1);
    up(&resource_2);
}
```

(a)

```
semaphore resource_1;
semaphore resource_2;
void process_A(void){
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}
void process_B(void){
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}
```

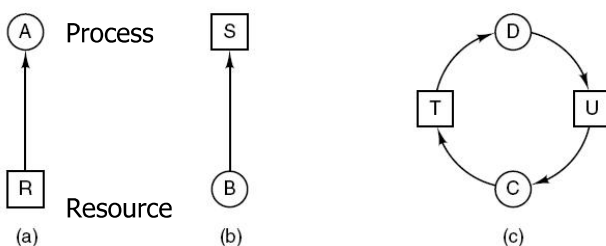
(b)

leexudong@nankai.edu.cn

10

Deadlock Model

SRAG(System Resource Allocation Graph)

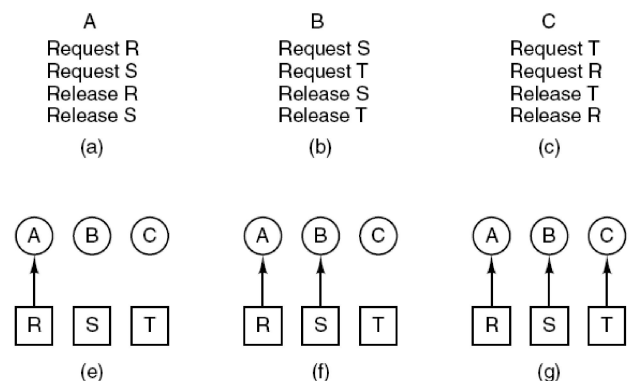


SRAG=(V,E)

leexudong@nankai.edu.cn

11

Deadlock Model

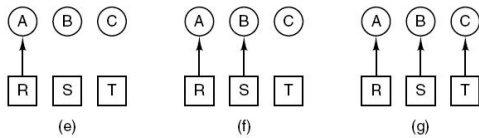


leexudong@nankai.edu.cn

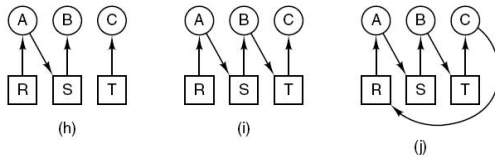
12

Deadlock Model

1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requests T
6. C requests R



Deadlock!

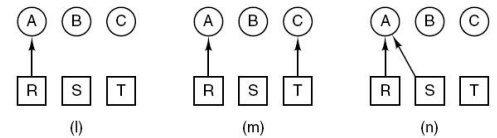


leexudong@nankai.edu.cn

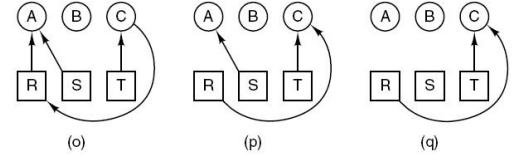
13

Deadlock Model

1. A requests R
2. C requests T
3. A requests S
4. C requests R
5. A releases R
6. A releases S



No Deadlock!



leexudong@nankai.edu.cn

14

Necessary Conditions

- Coffman et al., 1971
 - Mutual exclusion
 - Hold and wait
 - No preemption
 - Circular wait

leexudong@nankai.edu.cn

15

deadlock depend on a policy

- ?one resource can be shared among multi-procceses
- ?a process which holds a resource requests another resource
- ?policy which allows circular wait
- ?multi-processes concurrent

leexudong@nankai.edu.cn

16

Strategies for Deadlocks

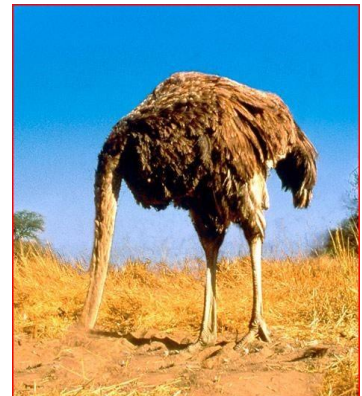
- Strategies for dealing with deadlocks:
 - Detection and recovery
 - let deadlocks occur, detect them, take action.
 - Dynamic avoidance
 - by careful resource allocation
 - Prevention
 - by structurally negating one of the four required conditions
 - ...

leexudong@nankai.edu.cn

17

Strategies for Deadlocks

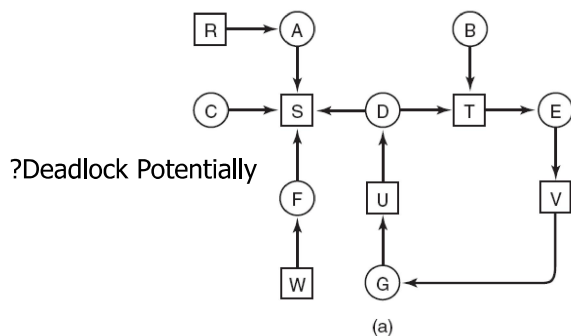
- the ostrich algorithm **Just ignore the problem**



leexudong@nankai.edu.cn

18

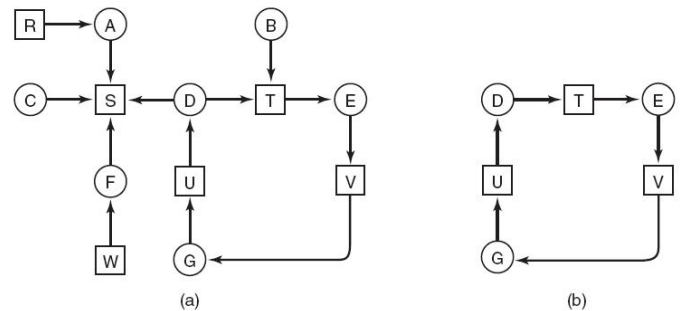
Deadlock Detection with One Resource of Each Type (1)



leexudong@nankai.edu.cn

19

Deadlock Detection with One Resource of Each Type (2)



leexudong@nankai.edu.cn

20

Deadlock Detection with One Resource of Each Type

Algorithm for detecting deadlock:

1. For each node, N in the graph, perform the following five steps with N as the starting node.
2. Initialize L to the empty list, designate all arcs as unmarked.
3. Add current node to end of L, check to see if node now appears in L two times. If it does, graph contains a cycle (listed in L), **algorithm terminates**.
- ...

leexudong@nankai.edu.cn

21

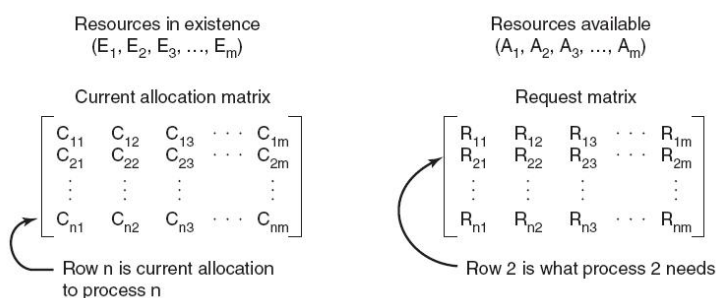
Deadlock Detection with One Resource of Each Type

4. From given node, see if any unmarked outgoing arcs. If so, go to step 5; if not, go to step 6.
5. Pick an unmarked outgoing arc at **random and mark** it. Then follow it to the new current node and go to step 3.
6. If this is initial node, graph does not contain any cycles, **algorithm terminates**. Otherwise, dead end. Remove it, go back to previous node, make that one current node, go to step 3.

leexudong@nankai.edu.cn

22

Deadlock Detection with Multiple Resources of Each Type (1)



identical equation:

$$\sum_{i=1}^n C_{ij} + A_j = E_j$$

leexudong@nankai.edu.cn

23

Deadlock Detection with Multiple Resources of Each Type (2)

Deadlock detection algorithm:

1. Look for an unmarked process, P_i , for which the i-th row of R is less than or equal to A.
2. If such a process is found, add the i-th row of C to A, mark the process, and go back to step 1.
3. If no such process exists, the algorithm terminates.

leexudong@nankai.edu.cn

24

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

Request matrix

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

- Recovery through preemption
- Recovery through rollback
- Recovery through killing processes

Process B



If ...

Free: 3
(a)

	2	

Free: 2
(b)

	4	
--	---	--

Free: 0
(c)

3	—
---	---

Free: 4
(d)

Free: 3
(a)

Free: 1
(b)

Free: 5
(c)

Free: 0
(d)

Free: 7
(e)



The Banker's Algorithm for a Single Resource

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)

	Has	Max
A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

leexudong@nankai.edu.cn

31

The Banker's Algorithm for Multiple Resources (1)

	Process	Tape drives	Plotters	Printers	CD ROMs
A	3	0	1	1	
B	0	1	0	0	
C	1	1	1	0	
D	1	1	0	1	
E	0	0	0	0	

Resources assigned

	Process	Tape drives	Plotters	Printers	CD ROMs
A	1	1	0	0	
B	0	1	1	2	
C	3	1	0	0	
D	0	0	1	0	
E	2	1	1	0	

Resources still needed

E = (6342)
P = (5322)
A = (1020)

leexudong@nankai.edu.cn

32

The Banker's Algorithm for Multiple Resources (2)

Algorithm for checking to see if a state is safe:

1. Look for row, R, whose unmet resource needs $\leq A$. If no such row exists, system will eventually deadlock since no process can run to completion
2. Assume process of row chosen requests all resources it needs and finishes. Mark process as terminated, add all its resources to the A vector.
3. Repeat steps 1 and 2 until either all processes marked terminated (initial state was safe) or no process left whose resource needs can be met (there is a deadlock).

udong@nankai.edu.cn

33

Deadlock Prevention

- Attacking the mutual exclusion condition
- Attacking the hold and wait condition
- Attacking the no preemption condition
- Attacking the circular wait condition

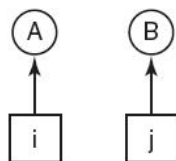
leexudong@nankai.edu.cn

34

Attacking the Circular Wait Condition

1. Imagesetter
2. Scanner
3. Plotter
4. Tape drive
5. CD-ROM drive

(a)



(b)

Approaches to Deadlock Prevention

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically

leexudong@nankai.edu.cn

35

leexudong@nankai.edu.cn

36

Order resources numerically

- $R_i (1, \dots, n)$
- $f(R_i)$: the id of R_i
- How to prove
 - If deadlock, so circular wait
 - i.e. existed:
 - $f(R_1) < f(R_2) < \dots < f(R_n) < f(R_1)$
 - **contradiction**

leexudong@nankai.edu.cn

37

Other Issues

- Two-phase locking
- Communication deadlocks
- Livelock
- Starvation

leexudong@nankai.edu.cn

38

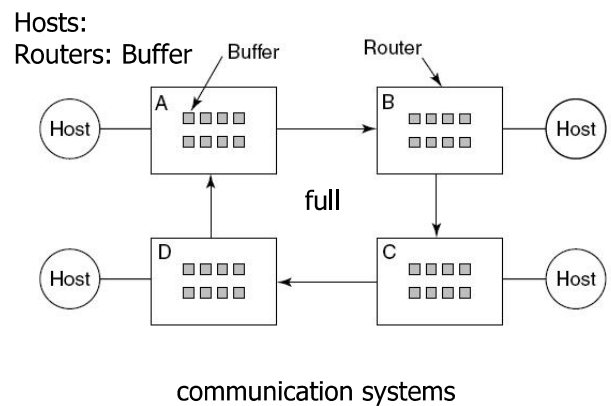
Two-phase locking

- In the first phase, the process tries to lock all the records it needs, one at a time.
 - If it succeeds, it begins the second phase, performing its updates and releasing the locks. No real work is done in the first phase.
- If during the first phase, some record is needed that is already locked
 - the process just releases all its locks and starts the first phase all over.

leexudong@nankai.edu.cn

39

Communication deadlocks



leexudong@nankai.edu.cn

40

Livelock

Case: Each one needs two resources and they use the **polling** primitive **enter_region** to try to acquire the necessary locks. If the attempt fails, the process just **tries again**.

```
void process_A(void) {
    enter_region(&resource_1);
    enter_region(&resource_2);
    use_both_resources( );
    leave_region(&resource_2);
    leave_region(&resource_1);
}

void process_B(void) {
    enter_region(&resource_2);
    enter_region(&resource_1);
    use_both_resources( );
    leave_region(&resource_1);
    leave_region(&resource_2);
}
```

Busy waiting that can lead to livelock

leexudong@nankai.edu.cn

41

Starvation

- Starvation
 - In a dynamic system, requests for resources happen all the time.
 - Some policy is needed to make a decision about who gets which resource when.
 - This policy, although seemingly reasonable, may lead to some processes never getting service even though they are not deadlocked.
- Case:
 - SJF

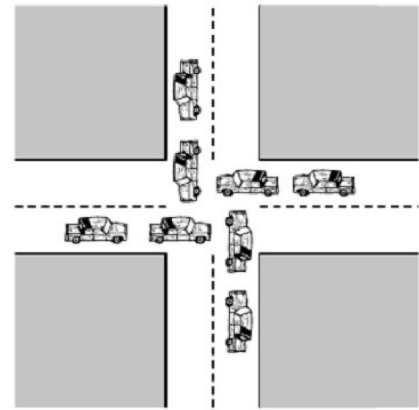
leexudong@nankai.edu.cn

42

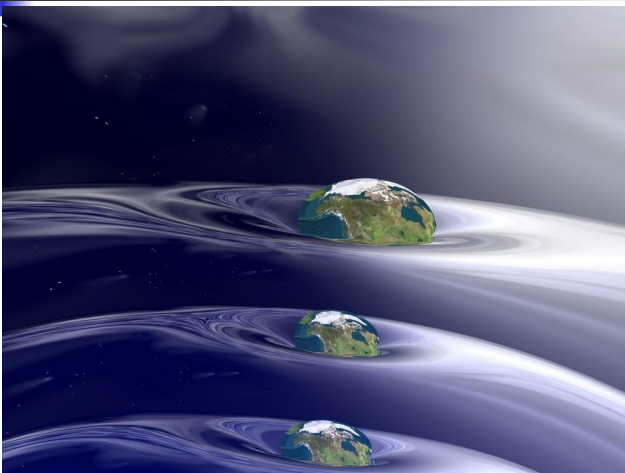
Deadlock, livelock, starvation

critical section	A section of code within a process that requires access to shared resources and which may not be executed while another process is in a corresponding section of code.
deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
livelock	A situation in which two or more processes continuously change their state in response to changes in the other process(es) without doing any useful work.
mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.

?Deadlock



(Optional) Parallel Universes



leexudong@nankai.edu.cn

45

Summary

- Deadlock Definition
- Deadlock Conditions
- Deadlock Modeling
- Deadlock Detection
- Deadlock Recovery
- Deadlock Avoidance
- Deadlock Prevention

leexudong@nankai.edu.cn

46

Q&A?

